

Python Fundamentals

Objectives

- Assign values to variables.

This is a note that you can only see in presenter view

Questions

- What basic data types can I work with in Python?
- How can I create a new variable in Python?
- How do I use a function?
- Can I change the value associated with a variable after I create it?
- How can I get help while learning to program?

Variables

Any Python interpreter can be used as a calculator:

```
3 + 5 * 4
```

23

This is great but not very interesting.

Variable assignment

To do anything useful with data, we need to assign its value to a *variable*. In Python, we can **assign** a value to a **variable**, using the equals sign `=`. For example, we can track the weight of a patient who weighs 60 kilograms by assigning the value 60 to a variable `weight_kg`:

```
weight_kg = 60
print(weight_kg)
```

60

From now on, whenever we use `weight_kg`, Python will substitute the value we assigned to it. In layperson's terms, **a variable is a name for a value**.

Variable names In Python:

variable names:

- can include letters, digits, and underscores
- cannot start with a digit
- are **case sensitive**.

This means that, for example:

- `weight0` is a valid variable name, whereas `0weight` is not
- `weight` and `Weight` are different variables

Types of data

Python knows various types of data. Three common ones are:

- integer numbers
- floating point numbers, and
- strings.

In the example above, variable `weight_kg` has an integer value of 60.

If we want to more precisely track the weight of our patient, we can use a floating point value by executing:

```
weight_kg = 60.3
print(weight_kg)
```

60.3

To create a string, we add single or double quotes around some text. To identify and track a patient throughout our study, we can assign each person a unique identifier by storing it in a string:

```
patient_id = '001'
print(patient_id)
```

001

Using Variables in Python

Once we have data stored with variable names, we can make use of it in calculations. We may want to store our patient's weight in pounds as well as kilograms:

```
weight_lb = 2.2 * weight_kg
print(weight_lb)
```

132.66

We might decide to add a prefix to our patient identifier:

```
patient_id = 'inflam_' + patient_id
print(patient_id)
```

inflam_001

Built-in Python functions

To carry out common tasks with data and variables in Python, the language provides us with several built-in [functions](#). To display information to the screen, we use the `print` function:

```
print(weight_lb)
```

```
132.66
```

```
print(patient_id)
```

```
inflam_001
```

Parentheses

When we want to make use of a function, referred to as calling the function, we follow its name by parentheses. The parentheses are important: if you leave them off, the function doesn't actually run! Sometimes you will include values or variables inside the parentheses for the function to use. In the case of `print`, we use the parentheses to tell the function what value we want to display. We will learn more about how functions work and how to create our own in later episodes.

Combining outputs using “print”

We can display multiple things at once using only one `print` call:

```
print(patient_id, 'weight in kilograms:', weight_kg)
```

```
inflam_001 weight in kilograms: 60.3
```

```
print("hello world")
```

```
[1] "hello world"
```

Nested Functions

We can also call a function inside of another [function call](#). For example, Python has a built-in function called `type` that tells you a value's data type:

```
print(type(60.3))
```

```
<class 'float'>
```

```
print(type(patient_id))
```

```
<class 'str'>
```

Nested Print

Moreover, we can do arithmetic with variables right inside the `print` function:

```
print('weight in pounds:', 2.2 * weight_kg)
```

```
weight in pounds: 132.66
```

Note: The above command, however, did not change the value of `weight_kg`:

```
print(weight_kg)
```

```
60.3
```

Updating variables

To change the value of the `weight_kg` variable, we have to **assign** `weight_kg` a new value using the equals = sign:

```
weight_kg = 65.0  
print('weight in kilograms is now:', weight_kg)
```

```
weight in kilograms is now: 65.0
```

Getting Help

Use the built-in function `help` to get help for a function. Every built-in function has extensive [documentation that can also be found online](#).

Help Example

```
help(print)
```

Help on built-in function print in module builtins:

```
print(...)  
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)  
  
    Prints the values to a stream, or to sys.stdout by default.  
    Optional keyword arguments:  
    file: a file-like object (stream); defaults to the current sys.stdout.  
    sep:  string inserted between values, default a space.  
    end:  string appended after the last value, default a newline.  
    flush: whether to forcibly flush the stream.
```

This help message (the function’s “docstring”) includes a usage statement, a list of parameters accepted by the function, and their default values if they have them.

Error messages

It is normal to encounter error messages while programming, whether you are learning for the first time or have been programming for many years. [We will discuss error messages in more detail later](#). For now, let’s explore how people use them to get more help when they are stuck with their Python code.

Task 1

Search the internet: paste the last line of your error message or the word “python” and a short description of what you want to do into your favourite search engine and you will usually find several examples where other people have encountered the same problem and came looking for help.

Hints

[StackOverflow](#) can be particularly helpful for this: answers to questions are presented as a ranked thread ordered according to how useful other users found them to be.

Take care: copying and pasting code written by somebody else is risky unless you understand exactly what it is doing!